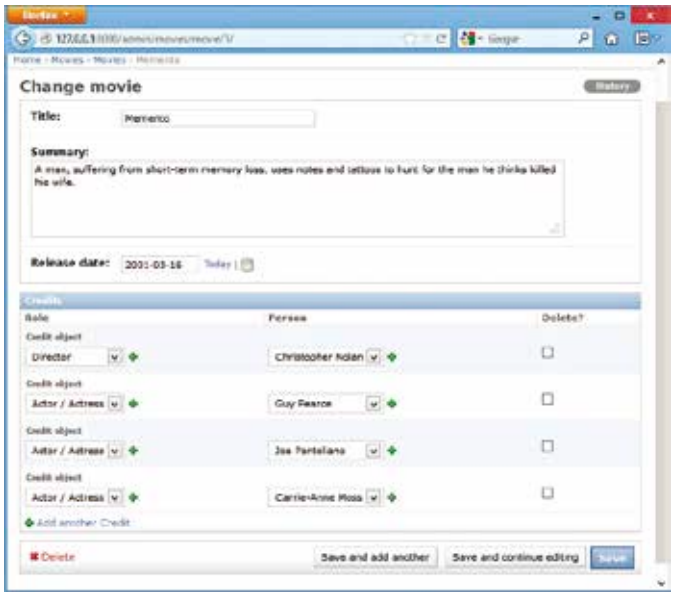
Check out a wearable Arduino
The Adafruit Gemma is a 1-inch wearable chip, based on the Arduino platform and is powered by the ATtiny85 with 3 I/O pins and progammable using Micro USB

PS4 may be > Xbox 720
According to a website, sthe Playstation 4 will have a run-capability of 1.84 teraflops, more than the Xbox 720's 1.23 teraflops **http://dgit.in/10u3HLH**



This is a much better way of adding movie credits. By the way you can click on the "+" to quickly add an object of that type!

what we need is an inline model, i.e. a way to input information about one model from within another model. Here is how that could be coded:

```
from movies.models import Credit, Movie, Person, Role
from django.contrib import admin
class CreditInline(admin.TabularInline):
        model = Credit
        extra = 3
class MovieAdmin(admin.ModelAdmin):
        inlines = [CreditInline]
admin.site.register(Movie, MovieAdmin)
admin.site.register(Person)
admin.site.register(Role)
```

The `CreditInline` class uses the `Credit` model, and allows for it to be embedded inline within another admin model. We are subclassing from `TabularInline`, but there is another option, `StackedInline` which looks a bit different; try it out. By setting a value for `extra` we tell the admin panel to allow people to enter three entries for Credits for a movie by default, although they are free to add more.

In the previous example we just used the `admin.site.register` to register our models with the admin. This uses the default settings for the admin panel. In this example we are overriding the settings for the admin panel for movie by making a class for it and customising its settings. The only setting we change here is that it should have an inline model, the `CreditInline` model.

If you run the site now, you will see a much better interface for adding movie credits. If we want, we can add the same inline option for people, so each person's admin page will list the movies that person has contributed to. For that we just add a `PersonAdmin` mirroring what we did for `MovieAdmin`, and register it the same we do for `Movie`. Add:

```
class PersonAdmin(admin.ModelAdmin):
        inlines = [CreditInline]
```

And modify the line `admin.site.register(Person)` to `admin.site.register(Person, PersonAdmin)`.

A little for the front

All we can do right now is add data, but there is no way to show it to users. Let's do something about that.

The `urls.py` file comes into play here. When someone visits a URL on our site, Django checks if the URL matches a pattern in this file, and calls the corresponding function. The pattern matching is done using Regex, and the functions are stored in the `views.py` file. Look at the examples in the `urls.py` file, then let's add one of our own. Above the admin code, add this:

```
url(r'^movie/(?P<movie_id>\d+)/$', 'movies.views.movie_detail'),
```

The first parameter to URL is a regular expression for matching a URL that has "movie/" followed by a number, which is captured as `movie_id`. It will then pass this movie_id to the `movie_detail` function in the views.py files in the movies app. Open the `views.py` file and add the following:

```
from movies.models import Credit, Movie
from django.http import HttpResponse
def movie_detail(request, movie_id):
    the_movie = Movie.objects.get(id=movie_id)
    credits = Credit.objects.filter(movie=the_movie)
    html = "<h1>%s</h1>" % (the_movie.title)
    for credit in credits:
        html += "%s: %s <br />" % (credit.role, credit.person)
    return HttpResponse(html)
```

If you visit the URL http://127.0.0.1:8000/movie/1/ the above function will be called with the value of `movie_id` set to `1`. What you do in this function is up to you, but it needs to return something for the browser to show. Here it is an `HttpResponse` with some HTML code.

You will see Django ORM (Object-Relation mapping) API used for the first time here. The first line retrieves the `Movie` object for the requested id. We then filter all the `Credit` objects that match this `Movie`. The movie name and credits and then put in some basic HTML code and returned to the browser. Add a few movies with credits and try this out to see a very simple HTML page.

Conclusion

And there you have it, a custom CMS with custom data. It is easy to apply this same method to any kind of data structure and work with books, libraries and authors, or music, artists, genres, albums and producers. You define the kind of data you want to work with, and Django makes you a rich, easy-to-use admin panel and the rest is up to you.

Ther is a lot more to Django, the admin panel can be customised a lot more, ther are powerful tools and features for dealing with the frontend using a simple template language. And there's a lot more to Django's database API which we just glossed over above. If you are intrigued visit www.djangoproject.com for a lot more information.